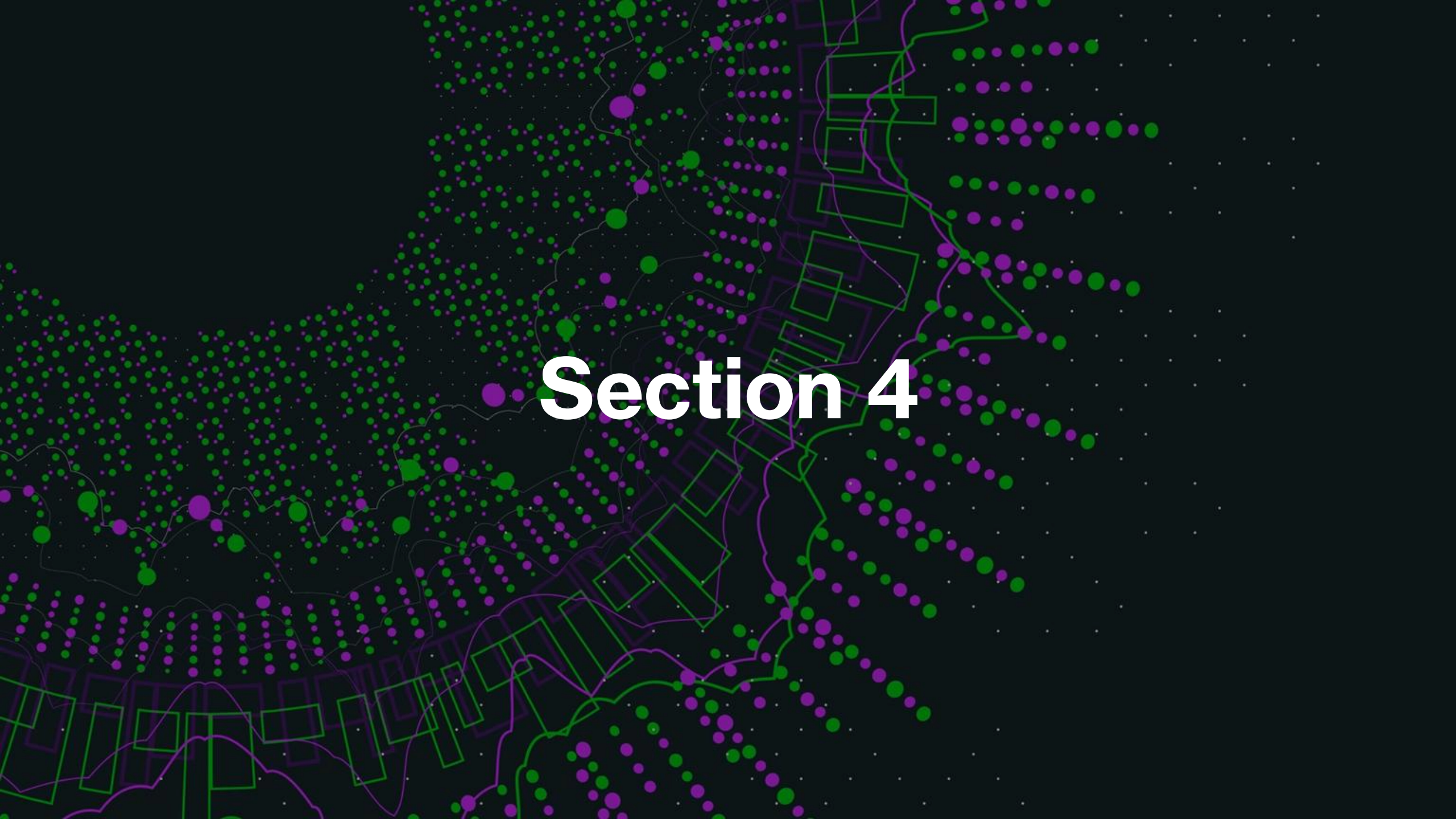


The background is a complex abstract pattern on a black field. It features numerous small green and purple dots scattered across the frame. Overlaid on these dots are thin, wavy green and purple lines. Additionally, there are several green-outlined rectangles of various sizes, some of which are nested or overlapping. The overall effect is a dense, layered, and somewhat chaotic visual texture.

Section 4

Section Overview

- Containerize a project using Docker
- Compose a project using containerization using Docker
- AWS account setup and AWS CLI configuration
- Manual deployment using EC2
- Automating Deployment using GitHub Actions



Containerization

Porting a project into containers using Docker

What is Containerization?

- Packaging software code
- Package only what is required to run the code.
- The Operating System libraries and dependencies
- Outcome:
 - Single, lightweight, executable unit
 - Operable on any infrastructure
- This eliminates environment-specific bugs (Windows OS -> MacOS)

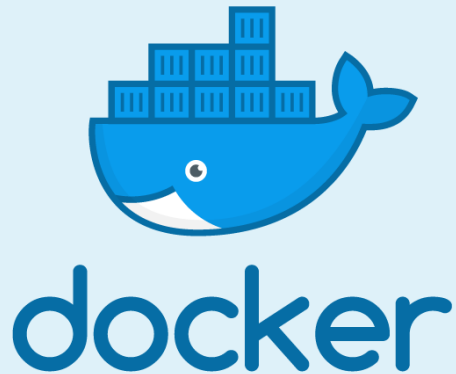
How does it work?

- Instead of traditional VM's, containers share the host machine's operating system kernel
- The Blueprint
 - Configuration file that specifies that application's environment and dependencies
- The Image
 - File used to build a static, immutable image
- The Container
 - The running instance of "The Image"

Why should we containerize?

- Benefits:
 - Portability
 - These applications run everywhere
 - On a local laptop, a testing server, or the cloud
 - Resource efficient
 - These containers only consume a fraction of the memory, CPU and storage that most VM's require
 - Deployment
 - They can be rerun and shut down in seconds.

Framework



- Open-source platform
- Most popular
- Total consistency
- Easy isolation
- Speed and scalability
- Dockerizing
- Uses a Dockerfile:
 - Each service gets their own Dockerfile in their respective folders

What exactly are we Dockerizing?

- Frontend
 - Vue -> Node
 - Dependencies -> node_modules
 - Port 5173
- Backend
 - Python v3.11
 - Dependencies -> Flask, Flask-cors, etc
 - Port 5000

Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```

Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```

Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```

Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```

Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```

Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```


Skeleton Code



```
# Base image
FROM <base-image>

# Set working directory
WORKDIR /app

# Copy files into the container
COPY <source> <destination>

# Run commands during image build
RUN <command>

# Expose a port (optional)
EXPOSE <port>

# Set environment variables (optional)
ENV <variable>=<value>

# Default command when container starts
CMD [<executable>, "<arg1>", "<arg2>"]
```

Containerizing the services



```
# Node image
FROM node:20

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependancies
RUN npm install

# Copy frontend source code
COPY . .

# Expose port for Vite
EXPOSE 5173

# Start development server
CMD [ "npm", "run", "dev", "--", "--host" ]
```



```
# Python image
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1

# Set working directory
WORKDIR /app

# Command to install Python Packages
RUN pip install --no-cache-dir flask flask-cors

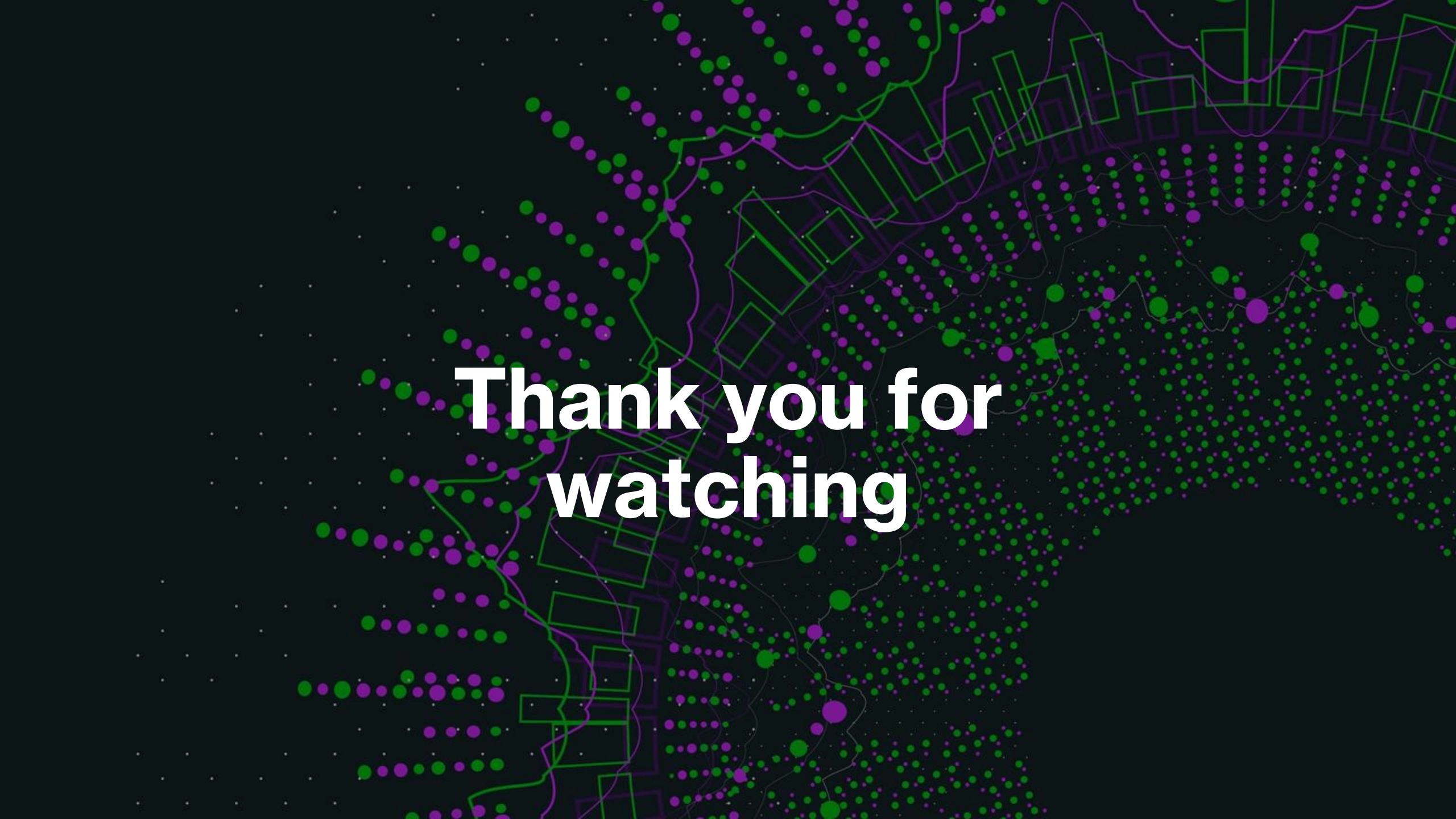
# Copy backend source code
COPY . .

# Expose flask port number
EXPOSE 5000

# Run application
CMD [ "python", "-m", "backend.app" ]
```

Are we done?

- Services are containerized ✓
- Ports are exposed ✓
- Can the containers communicate with each other...
- No.

The background is a black field filled with a complex pattern of green and purple dots, lines, and rectangles. The dots are arranged in various clusters and trails. The lines are thin and wavy, connecting some of the dots. The rectangles are thin and appear to be floating or overlapping the other elements. The overall effect is a dense, abstract, and somewhat chaotic visual field.

**Thank you for
watching**